

Deliverable 2 — Function F6: Real Beta Function $B(x, y)$

From-Scratch Implementation, Tkinter GUI, Repository, Updated Requirements

Shoban Bhowmik (ID 40325778)

SOEN 6011 — Software Engineering Processes — Section CC

2026-07-24 (v0.2.0)

- 1 What D2 adds
- 2 From-scratch boundary (P5)
- 3 Elementary functions
- 4 Architecture & exceptions
- 5 Tkinter GUI & errors
- 6 Verification (P5)
- 7 Repository (P6)
- 8 Requirements v0.2 (P7) & GAI (C-03)

What Deliverable 2 adds

Starting point (D1): Algorithm B — $B(x, y) = \exp(\ln \Gamma(x) + \ln \Gamma(y) - \ln \Gamma(x+y))$, a hand-written Lanczos $\ln \Gamma$, shipped as a CLI that borrowed only $\ln, \exp, \sin, \pi$ from `math`.

- **Problem 5 [60]** — reimplement *from scratch* (no math library) + a **Tkinter GUI**, with exception handling and helpful errors.
- **Problem 6 [20]** — public Git repository, high-quality commits, README.
- **Problem 7 [20]** — update the requirements from the D2 build.

Domain $x > 0, y > 0$ (D-001). Individual work; CASTROFF GAI prompts logged (C-03).

Problem 5 — the “from scratch” boundary (D-008)

Permitted: input, output, arithmetic, UI. Prohibited: every other library function. The whole mathematical dependency was just four primitives:

Borrowed (D1)	Role	Replacement (elementary.py)
<code>math.log</code>	logarithm	<code>ln</code> : mantissa reduction + <code>atanh</code> series
<code>math.exp</code>	exponential	<code>exp</code> : $k \ln 2 + r$ reduction + Taylor
<code>math.sin</code>	reflection	<code>sin</code> : argument reduction + Taylor
<code>math.pi</code>	constant	PI: correctly-rounded literal

No `math.gamma/lgamma/beta` — ever. **IMPL-01** scans the shipped source and fails on any `math/numpy/scipy` import: it passes.

The elementary functions: reduce $\rightarrow$ series $\rightarrow$ rebuild

```
def ln(x): # x = m * 2**e, m in [1/sqrt2, sqrt2)
    e = 0; m = x
    while m >= _SQRT2: m *= 0.5; e += 1
    while m < _SQRT1_2: m *= 2.0; e -= 1
    s = (m-1)/(m+1); s2 = s*s # |s| <= 0.1716 -> fast series
    term = s; total = s; denom = 1
    while True: # ln(m) = 2*(s + s^3/3 + s^5/5 + ...)
        term *= s2; denom += 2; d = term/denom; total += d
        if abs(d) < _EPS*abs(total)+_EPS: break
    return e*LN2 + 2*total
```

Verified vs. a trusted oracle: worst error $\exp \leq 6 \times 10^{-15}$, $\ln \leq 6 \times 10^{-16}$, $\sin \leq 9 \times 10^{-16}$. Every loop is bounded (REL-03).

Separation (testable core):

- `elementary.py` — primitives
- `beta_core.py` — pure $\ln \Gamma, \beta$
- `validation.py` — parsing + messages
- `gui.py` — Tkinter only

Typed errors: `BetaError`

- `InputError`
(empty/non-numeric/non-finite)
- `DomainError`, `RangeError`
- `NumericalRangeError`

Catch categories separately (ERR-02); one except `BetaError` guarantees no traceback (REL-01). The core raises `NumericalRangeError` instead of returning `inf/nan` (REL-02).

Problem 5 — the Tkinter GUI

Beta Function Calculator v0.2.0

Beta Function $B(x, y)$

Computes $B(x, y) = \Gamma(x)\Gamma(y) / \Gamma(x+y)$ for $x > 0, y > 0$.

Inputs

x (must be > 0):

y (must be > 0):

Supported range: $0 < x, y \leq 10000$. Example: $x = 2, y = 3$.

Calculate Clear Help

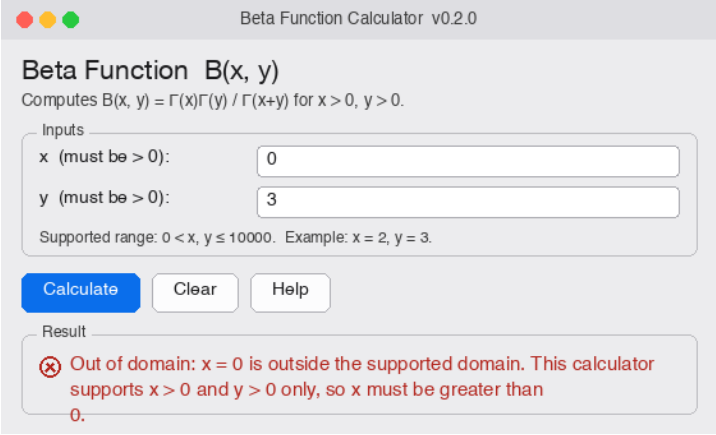
Result

✓ Result: $B(2, 3) = 0.0833333$ (to 6 significant figures)

Wireframed first (D-010), traced to persona “Maya”.

- Labels state the domain: “x (must be > 0)”
- Keyboard-first: focus, Tab, Enter=calc, Esc=clear, F1=help (UI-01)
- Status is *text* (a word + glyph), colour only redundant (ACC-03)
- Runs via `python3 src/gui.py` — no IDE (POR-01)

Helpful errors, never a traceback (ERR-01, REL-01)



The screenshot shows a window titled "Beta Function Calculator v0.2.0". Inside, the title "Beta Function B(x, y)" is followed by the formula $B(x, y) = \Gamma(x)\Gamma(y) / \Gamma(x+y)$ for $x > 0, y > 0$. There are two input fields: "x (must be > 0):" with the value "0" and "y (must be > 0):" with the value "3". Below these is a note: "Supported range: $0 < x, y \leq 10000$. Example: $x = 2, y = 3$." There are three buttons: "Calculate" (highlighted in blue), "Clear", and "Help". Below the buttons is a "Result" section containing a red error message: "⊗ Out of domain: x = 0 is outside the supported domain. This calculator supports $x > 0$ and $y > 0$ only, so x must be greater than 0."

Each expected failure — empty, non-numeric, out-of-domain, out-of-range, numerical — gets its own specific, corrective message. (Figures are layout renders of `src/gui.py`, verified by GUI-01...06; the live window is shown in the demo.)

- **IMPL-01** shipped source imports no math library.
- **ACC-01** 18 reference values, worst relative error 1.17×10^{-14} — *identical to D1*: the from-scratch build cost nothing in accuracy.
- **VAL/ERR** five validation paths raise the right typed exception + helpful message.
- **REL-02** 36 large/small inputs all finite; **REL-03** bounded loops.
- **PERF-01** 7.8×10^{-3} ms/call; **POR-01** subprocess GUI launch.
- **GUI-01...06** handlers report the correct status for valid + each invalid case.

Matrix: docs/verification_matrix_d2.md

Problem 6 — repository, commits, README

- Public Git host; URL recorded in the README (published from the author's account).
- **Incremental, high-quality history** (D-013): one cohesive change per commit, imperative subjects (“Add from-scratch elementary functions”, “Add Tkinter GUI over the pure core”) — not a single bulk upload.
- `.gitignore` excludes byte-code, `.DS_Store`, $\text{\LaTeX}$ artefacts, IDE settings, zips.
- `README.md`: purpose, exact run command, examples + screenshots, error guide, structure, version — a new user can run it from the README alone.

Problem 7 — requirements v0.1 $\rightarrow$ v0.2 (D-012)

All v0.1 IDs kept; changes discovered *through implementation*:

Change	Reason
FR/USE/POR reworded	CLI $\rightarrow$ Tkinter GUI
VAL-02	also reject <code>inf/nan</code>
ACC-03 promoted $S \rightarrow M$	accessibility is first-class
REL-02 explicit	raise <code>NumericalRangeError</code>
+ IMPL-01	from-scratch mandate
+ UI-01	keyboard operability
+ ERR-02	typed input vs numerical errors

24 requirements; nothing deleted (superseded wording preserved). A-04/A-05 re-validated.

Tool: Claude (Opus 4.8) via Claude Code. One CASTROFF prompt per problem (TASK / RESTRICTIONS / OUTPUT FORMAT / AUDIENCE).

- **P5:** generative + transformational — boundary, primitives, GUI. *Verified:* harness proved no library import + machine-precision primitives.
- **P6:** advisory — .gitignore, commit plan, README. *Adjusted:* publishing left to the author's account (outward-facing).
- **P7:** transformational + review — evolved the baseline; each change tied to a matrix check + decision.

Every output critically evaluated (accepted/adjusted/rejected) and independently verified. Logs: docs/prompts/gai_prompt_log.md.

Limitations & summary

Honest limitations: domain $x, y > 0$ only; magnitude cap 10^4 ; very large near-equal inputs underflow to 0.0 (correct finite double); accuracy bounded by Lanczos $g=7$; figures are renders (live window in demo).

Summary: the Beta function is implemented *from scratch* behind a Tkinter GUI with typed exceptions and helpful messages; published with an incremental history and a README; requirements updated to v0.2. **27/27** checks pass; accuracy unchanged at 1.17×10^{-14} .

Demo: `python3 src/gui.py` — $B(2, 3)$, $B(0.5, 0.5)=\pi$, then $x=0$, $x=\text{"abc"}$, $x=10^6$.

References

 ISO/IEC/IEEE 29148:2018 — Requirements engineering.

 C. Lanczos, “A Precision Approximation of the Gamma Function,” 1964.

 J.-M. Muller, *Elementary Functions: Algorithms and Implementation*, 3rd ed., 2016.

 W. Cody and W. Waite, *Software Manual for the Elementary Functions*, 1980.

 NIST Digital Library of Mathematical Functions, §5.12.

 Python Software Foundation, *tkinter* documentation.

Thank you — questions?